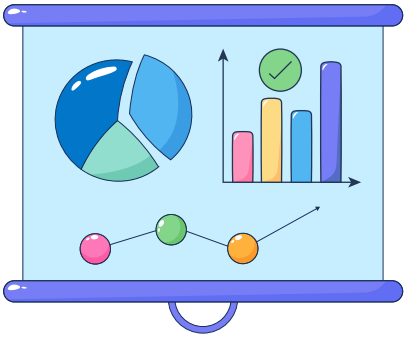
Publishing Custom Metrics and Logs in CloudWatch

Explore methods to publish custom metrics and logs in AWS CloudWatch, including using PutMetricData API, Embedded Metric Format, and the CloudWatch agent. Understand how to design namespaces, dimensions, and structured logging to gain actionable insights into your application's performance and reliability.

Let's say we deploy a Lambda function that processes customer orders. AWS will automatically give us some default metrics, such as invocations, errors, and duration. But what if our business needs to track the number of premium orders processed per hour or log payment gateway failures? That kind of context doesn't come built-in.



Instrumentation is adding telemetry into our application logic so that CloudWatch can help us answer meaningful questions about performance, reliability, and behavior under real-world conditions.

Ask

Publishing custom metrics in applications

By default, AWS automatically tracks system metrics like CPU usage, disk I/O, and network traffic for EC2. But suppose we want to track application-specific data such as the number of user sign-ups, items added to a cart, or the processing time of a specific business function. In that case, we need to publish custom metrics.

There are three main methods to publish custom metrics. Let's understand these methods.

Method 1: Using the `PutMetricData` API

The most straightforward way to publish custom metrics in an application is to use the `PutMetricData` API via an SDK or CLI. We simply construct a metric object, including `Namespace`, `MetricName`, `Dimensions`, and `Value`, and send it to CloudWatch.

The code snippet below shows how custom metrics can be published via the `PutMetricData` API in Python SDK (Boto3):

main.py Python

```python
import boto3

cloudwatch = boto3.client('cloudwatch')

response = cloudwatch.put_metric_data(
    Namespace='MyApp/Users',
    MetricData=[
        {
            'MetricName': 'SuccessfulSignups',
            'Dimensions': [
                {'Name': 'PlanType', 'Value': 'Premium'}
            ],
            'Value': 1
```

Ask

Python code to publish custom metrics using the PutMetricData API

This method, however, has one downside. Making frequent API calls can add latency and complexity to the application.

Method 2: Embedded metric format (EMF)

Embedded metric format (EMF) is the modern, recommended approach for applications that also generate logs, especially in serverless and containerized environments. It is a special JSON layout that includes our metric name, value, unit, dimensions, and namespace. In this method, we write the JSON object to our logs, and CloudWatch automatically scans these logs, finds the embedded metric data, and publishes it.

> **Note:** EMF combines logging and metric publication into a single, efficient operation.

Let's say our code processes orders, and we want to track how many are successful, how many fail, and how long each takes. We can log these details in Python with EMF as follows:

index.yaml YAML

```yaml
{
  "_aws": {
    "Timestamp": 1715580000000,
    "CloudWatchMetrics": [
      {
        "Namespace": "MyApp/Orders",
        "Dimensions": [["Region", "OrderType"]],
        "Metrics": [
          { "Name": "SuccessCount", "Unit": "Count" },
          { "Name": "FailureCount", "Unit": "Count" },
          { "Name": "ProcessingTime", "Unit": "Milliseconds" }
        ]
      }
```

Ask

Embedded metric format in Python

Let's understand the emitted metric:

- The `"_aws"` block contains metadata that CloudWatch uses to parse metrics.
- `"CloudWatchMetrics"` defines the namespace, dimensions, and metric names/units.
- The other fields, including `Region`, `OrderType, ProcessingTime`, are our actual metric data and dimension values.

This method is especially helpful when we're working in EC2, containers, or hybrid environments and not using a Lambda-specific library. But while it's powerful, it does involve more boilerplate and requires strict formatting discipline. If there is a tiny mistake, like a typo in a key name or an incorrect structure, CloudWatch will silently fail in creating the metric, which is difficult to debug.

To abstract away these complexities, AWS offers AWS PowerTools. Here's how it looks in Python using the `aws_lambda_powertools` package:

main.py C++

```
from aws_lambda_powertools import Logger, Metrics
from aws_lambda_powertools.metrics import MetricUnit

logger = Logger()
metrics = Metrics(namespace="MyApp/Checkout")

@metrics.log_metrics
def lambda_handler(event, context):
    metrics.add_metric(name="CheckoutDuration", unit=MetricUnit.Milli
    logger.info("Checkout completed successfully")
    return {"statusCode": 200}
```

Ask

Python code to publish logs using the aws_lambda_powetools package

Let's understand the code:

- **Line 5:** We declare a metric namespace (`MyApp/Checkout`) once.
- **Line 7:** The decorator `@metrics.log_metrics` ensures the metric data is logged in EMF format automatically.
- **Line 9:** The `add_metric()` function lets us specify what metric to emit and in what unit.
- **Line 10:** We can also log as usual (`logger.info(...)`) and even integrate with structured logging and tracing.

This method is perfect when we're building with Lambda and want clean code in Python, Node.js, or Java (current supported runtimes).

Method 3: CloudWatch agent

We can install CloudWatch agent on EC2, on-premises servers, or even in other cloud environments to collect system-level data and custom logs. It can push this data to CloudWatch, where we can treat it just like any native AWS telemetry. This agent is especially useful in hybrid or lift-and-shift scenarios where infrastructure visibility is critical.

Ask

How CloudWatch agent works

Adding dimensions and namespaces strategically

Publishing custom metrics effectively is more than just sending numbers. It's about designing them to answer specific questions about the application's health and performance. Let's discuss some optimized practices for effectively publishing custom metrics in our applications.

Use a clear naming convention

A consistent naming scheme prevents confusion. A good practice is to use a hierarchical structure for namespaces and descriptive metric names.

- **Namespace:** Group metrics by application or service, such as, `PaymentService` or `OrderProcessor`.
- **Metric name:** Be specific about what is measured, such as `SuccessfulTransactions`, `ProcessingTime` or `CacheHitRate`.

Choose strategic dimensions

When we emit custom metrics, think of dimensions as tags we use to filter the data later. A metric without dimensions is like a file without a folder. It's there, but hard to organize or filter.

In simple terms, choose dimensions that align with the business questions as follows:

- To see performance by customer tier, add a `Tier: "Premium"` dimension.
- To analyze failures by payment provider, add a `Provider: "Stripe"` dimension.
- To isolate issues in an A/B test, add a `Version: "B"` dimension.

Ask

> With these, we can easily compare the `ProcessingTime` for premium vs. standard tiers or see which payment provider has the highest error rate.

Publishing custom logs in applications

Logs provide detailed, contextual information about events occurring within the application or system. As discussed in the previous lesson, for services like AWS Lambda, API Gateway, Amazon ECS, and others, anything the application writes to standard output (`stdout`) or standard error (`stderr`) is automatically captured and sent to a corresponding CloudWatch Log Group.

There are several methods to publish custom logs in applications. Let's discuss these methods.

Method 1: CloudWatch agent

Just like custom metrics, we can publish custom metrics using the CloudWatch agent. In EC2, we'll typically install the CloudWatch agent or use the unified agent to push application logs into CloudWatch logs.

We can configure the CloudWatch agent to monitor specific log files (e.g., `/var/log/nginx/access.log` or `C:\MyApp\logs\app.log`). The agent streams new log entries from these files to CloudWatch Logs in near-real time.

Method 2: Using the `PutLogEvents` API

Similar to metrics, we can programmatically send logs to CloudWatch using the SDK. The application code batches log events and sends them directly to a specific log group and stream using the `PutLogEvents` API call.

main.py Pytho. Ask

```
import boto3
logs = boto3.client('logs')
```

```
logs.put_log_events(
    logGroupName='my-app-logs',
    logStreamName='api-errors',
    logEvents=[
        {
            'timestamp': int(time.time() * 1000),
            'message': 'User failed to authenticate'
        }
    ]
)
```

Python code to publish custom logs using the PutLogEvents API

We need to create the log group and log stream beforehand using `create_log_group` and `create_log_stream`.

> As discussed previously, Embedded Metric Format is used to extract metrics from logs. Therefore, we log structured JSON while using EMF.

Structured logging in applications

There's more to good logging than printing messages. Logs are most useful when they're:

- **Structured:** Outputted as JSON or key-value pairs for easy parsing.
- **Tagged:** Include user IDs, transaction IDs, or operation names.
- **Consistent:** Follow the same format across functions and services.

This kind of logging supports **CloudWatch Logs Insights.** It is a fully managed, interactive log analytics service that allows us to write SQL-like queries to search, analyze, and visualize log data in real time. It's designed specifically for developers and operators who need to troubleshoot, debug, and monitor applications running on AWS.

Ask

We'll discuss more about querying and analyzing CloudWatch logs ahead in the lesson. For now, let's focus on publishing logs in CloudWatch.

Let's start by considering a hypothetical situation where our ClouWatch logs look something like this:

index.yaml YAML

```
User login failed
Payment declined
Function timeout after 3000ms
```

Given these logs, how would we search for all failed login attempts from the user `abc123`? It would be difficult to scroll through all the logs. These logs are free-form text. There's no structure, no consistency, and no way to efficiently filter or analyze them.

Now imagine logs like this instead:

index.yaml YAML

```
{
  "level": "ERROR",
  "event": "LoginFailed",
  "userId": "abc123",
  "ip": "203.0.113.14",
  "timestamp": "2025-05-13T16:22:10Z"
```

Ask

```
}
```

Sample logs

This is structured logging. Here, logs are emitted as JSON or key-value pairs, with a consistent schema and meaningful fields. Every *log entry* is like a database row. We can query it, filter it, and correlate it with other entries.

Conclusion

Publishing custom metrics in CloudWatch allows developers to go beyond default telemetry and track application-specific behavior, such as business events or operational anomalies. With structured logging and embedded metric formats,

Ask